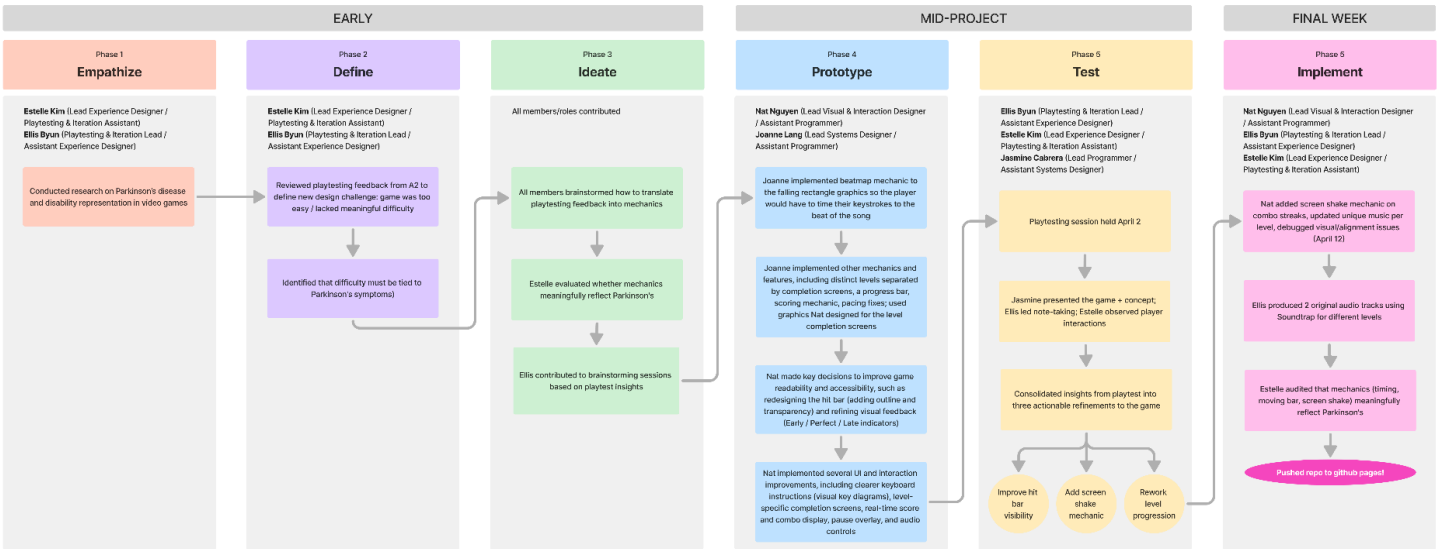
Process & Decision Documentation

Process Overview Visualization (Group Work Only)

https://www.figma.com/board/m9n2WSNvKWZ882ZpkUhfc1/Untitled?node-id=0-1&t=LKb7zbT8bUtQtfLb-1



Project/Assignment Decisions

- Estelle made the informed decision to make the game more difficult with the introduction of more mechanics based on feedback from the playtest expressing that the game was too easy. Utilizing research, AI brainstorming, as well as consulting from the group, the screen shake was ideated.
- Based on midterm playtesting feedback, Joanne implemented beatmap mechanic to the falling rectangle graphics so the player would have to time their keystrokes to the beat of the song
- Joanne implemented other mechanics and features, including distinct levels separated by completion screens, a progress bar, scoring mechanic, pacing fixes; used graphics Nat designed for the level completion screens
- After consulting with group and coming to agreement, Joanne decided to scrap the hold note logic altogether; game will only have single-tap mechanic for the falling notes

- Joanne consulted with group members after each major update to the game, gathering feedback from group members' playtesting
- Based on playtesting feedback, Nat focused on improving UI clarity and feedback systems, ensuring that players could better understand timing, controls, and overall gameplay flow
- Nat implemented several UI and interaction improvements, including clearer keyboard instructions (visual key diagrams), level-specific completion screens, real-time score and combo display, pause overlay, and audio controls
- Nat made key decisions to improve game readability and accessibility, such as redesigning the hit bar (adding outline and transparency) and refining visual feedback (Early / Perfect / Late indicators)
- Based on peer feedback, Nat introduced dynamic difficulty through a screen shake mechanic tied to player combo/streak
- Nat adjusted the level progression system, deciding to introduce the spacebar in Level 2 and move D and K to Level 3, creating a smoother learning curve for players
- Nat contributed to audio design decisions, including separating menu and gameplay music and ensuring consistent sound behavior across different game states
- Nat actively participated in manual testing and iteration, debugging gameplay issues (e.g., scoring inconsistencies, audio overlap, UI alignment) and refining the game based on repeated testing
- Nat regularly reviewed updates with the team and incorporated peer feedback into design and implementation decisions to improve overall game quality

Role-Based Process Evidence

Entry #1: Time-Based Graphics Logic & Further Refinement of Levels

Name: Joanne Lang

Role(s): Lead Systems Designer, Assistant Programmer

Primary responsibility for this work:

Goal of Work Session

- Finalizing 70-80% of game levels for A3 playtesting, including time-based logic for graphics and scoring mechanics

Tools, Resources, or Inputs Used

- ChatGPT 5.2 and ChatGPT 4.0
- Claude Sonnet 4.6
- Epidemic Sound for audio
- Prior drafts or code
- Playtesting feedback

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 28-29, 2026

Tool Disclosure: ChatGPT 5.2, ChatGPT 4.0, Claude Sonnet 4.6

Purpose of Use: Help with implementing specific features, debugging while coding, wording improvements and explanations of code snippets

Summary of Interaction: Code snippets for feature implementation, explanation of snippets, debugging help.

Human Decision Point(s): Probing further for more debugging help when initial output code wouldn't work; rejected method from one GenAI tool in favour of one from another GenAI tool that made more sense.

Integrity & Verification Note: Playtesting, general testing and debugging, re-reading assignment criteria, feedback from group, clarity/usability checks.

Scope of GenAI Use: GenAI contributed code snippets, some rewrites, explanations of code logic; I decided which parts of the code to use and implement in my own files.

Limitations or Misfires: For both ChatGPT and Claude, often couldn't rewrite files without breaking other parts of the code/messing up the ordering of lines; ChatGPT often provided output that wouldn't work and required further prompting.

Summary of Process (Human + Tool)

- Gave GenAI existing files for context and asked both ChatGPT and Claude for code to make the rectangles fall to the beat of the song
- ChatGPT said I needed p5.sound but Claude said I don't actually need it because I could use the HTML <audio> element, bgMusic.currentTime; I decided to go with Claude's implementation because it was explained better → accepted Claude's rewrite of sketch.js
- Continued using Claude to get code snippets to add an 8-count pause before rectangles start falling, implement the oscillating hit bar, randomize the falling rectangles
- Claude would sometimes break other parts of the code during full rewrites, resulting in the Play/Instructions buttons not working → asked for a solution to this and I implemented effectively (fixed ordering of code lines)
- Generated customizable beatmap for rectangles; did some heavier debugging when Play/Instructions broke again
- Asked what the purpose of levels.js was and told it to move the levels logic and beatmaps over from sketch.js for better organization
- Helped me scour my code to look for the handlers I was trying to edit
- Tried prompting for code to implement hold note logic, gave up after results that didn't work after testing and debugging

Decision Points & Trade-offs

- Tried prompting for code to implement hold note logic; tested and debugged for hours trying to get it to function properly → decided to scrap this idea of hold notes entirely due to lack of necessity and to prioritize more important parts
- Focused on refining and perfecting the existing single-tap note logic and rest of the game levels
- Ended up reducing unnecessary complexity

Verification & Judgement

Playtesting, general testing and debugging, re-reading assignment criteria, feedback from group, clarity/usability checks.

Limitations, Dead Ends, or Open Questions

- Some rewrites of code would break other parts of the code

- Had to scrap hold note feature because I couldn't get working code; keep it simple

Entry #2: UI Coding

Name: Nat Nguyen

Role(s): Lead Visual & Interaction Designer, Assistant Programmer

Primary responsibility for this work:

- Polish UI clarity and feedback systems
- Ensure accessibility and readability of visuals
- Maintain visual consistency across all 3 levels
- Align visuals with intended emotion and cyberpunk aesthetic
- Implement UI elements directly in code
- Debug visual alignment and gameplay-related issues

Goal of Work Session

- Enhance visual feedback systems to support player learning and timing accuracy
- Improve clarity of instructions and controls
- Refine UI elements for a polished and cohesive experience
- Integrate feedback systems (score, combo, pause, audio) into gameplay

Tools, Resources, or Inputs Used

- ClaudeAI Sonnet 4.6
- Cursor 3.0
- ChatGPT 5.2
- ChatBox GPT-4o
- Prior drafts of code
- Manual testing and iteration

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: April 1, 2026

Tool Disclosure: ChatGPT 5.2, Claude AI Sonnet 4.6, Cursor 3.0, ChatBox GPT-4o

Purpose of Use: Assist with implementing UI features, improving instruction clarity, and structuring gameplay feedback systems in code

Summary of Interaction:

Used GenAI tools to generate and refine UI-related code, including:

- Keyboard instruction diagram (F/J key visuals)
- Level complete screen UI (level-specific titles + star scoring)
- In-game UI (score, combo display, level indicator)
- Pause overlay and mute button functionality
- Audio system updates (menu vs gameplay music handling)

Human Decision Point(s):

- Ensured a consistent cyberpunk theme throughout UI implementation
- Decided how feedback (score, combo, stars) should be displayed
- Determined when and where UI elements appear
- Adjusted and integrated generated code into the existing system

Integrity & Verification Note:

All generated code was reviewed, tested, and modified by multiple group members to ensure correct functionality and alignment with the intended design.

Scope of GenAI Use:

Used as a support tool for implementation and iteration, not for complete system design. UI and design choices thought by Nat.

Limitations or Misfires:

- Some generated solutions caused conflicts (e.g., overlapping audio tracks)
- Required debugging and manual fixes for proper integration (inconsistent scoring)
- Styling suggestions sometimes needed adjustment to match the game aesthetic

Entry #3: Debugging, Polishing, Feedback Implementation

Name: Nat Nguyen

Role(s): Lead Visual & Interaction Designer, Assistant Programmer

Primary responsibility for this work:

- Debug visual alignment and gameplay-related issues
- Implement playtesting feedback into the game
- Increase gameplay difficulty through new mechanics
- Refine and polish overall user experience

Goal of Work Session

- Improve overall game clarity and usability based on peer feedback
- Introduce more engaging and dynamic difficulty
- Finalize gameplay systems for submission-quality polish

Tools, Resources, or Inputs Used

- Codex
- Playtesting feedback from peers
- Manual testing and iteration
- Prior drafts of code

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: April 12, 2026

Tool Disclosure: Codex 3.0

Purpose of Use: Assist with implementing gameplay changes, debugging issues, and refining mechanics based on playtesting feedback

Summary of Interaction:

Used Codex to support implementation of:

- Screen shake mechanic tied to combo/streak system
- Updated level progression (Spacebar in Level 2, D/K in Level 3)
- Adjustments to level music

Human Decision Point(s):

- Determined which feedback to prioritize and implement
- Designed how difficulty should scale (screen shake mechanic)

- Chose new key progression for improved learning curve
- Reviewed and adjusted all generated code before integration
- Final adjustments to gameplay based off human playtesting

Integrity & Verification Note: All outputs were tested, debugged, and refined to ensure correct functionality and alignment with gameplay goals

Scope of GenAI Use: Used for assisting implementation and debugging, while all design decisions and final integrations were finalized by Nat

Limitations or Misfires:

- Some generated code required debugging to fit my goals for the game (the screen shaking made the kitty and bunny graphics shake too, went in and isolated the screen shaking from the graphics)
- Audio and gameplay interactions occasionally conflicted and needed refinement

Entry #4: Playtest Note-taking & Music Production

Name: Ellis Byun

Role(s): Playtesting & Iteration Lead, Assistant Experience Designer

Primary responsibility for this work: Note-taking during group playtest session and producing 2 new songs for the different levels.

Goal of Work Session

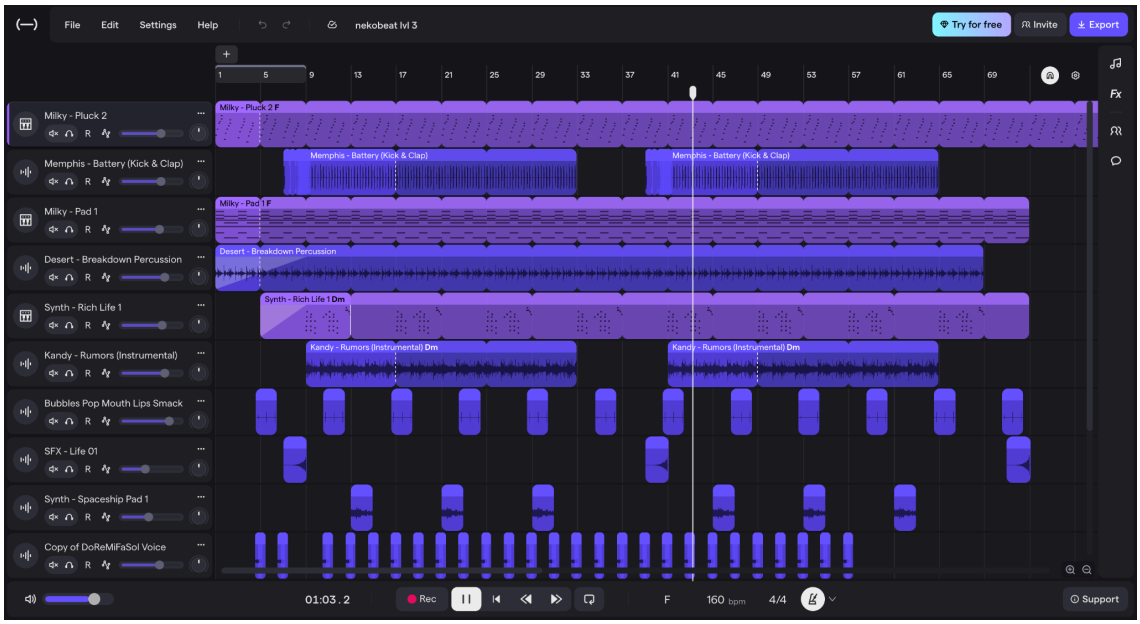
- Contributed to brainstorming sessions for potential modifications/additions based on playtesting insights
- Created 2 new audio tracks for the game using Soundtrap to support rhythm mechanics and have a different song for each level

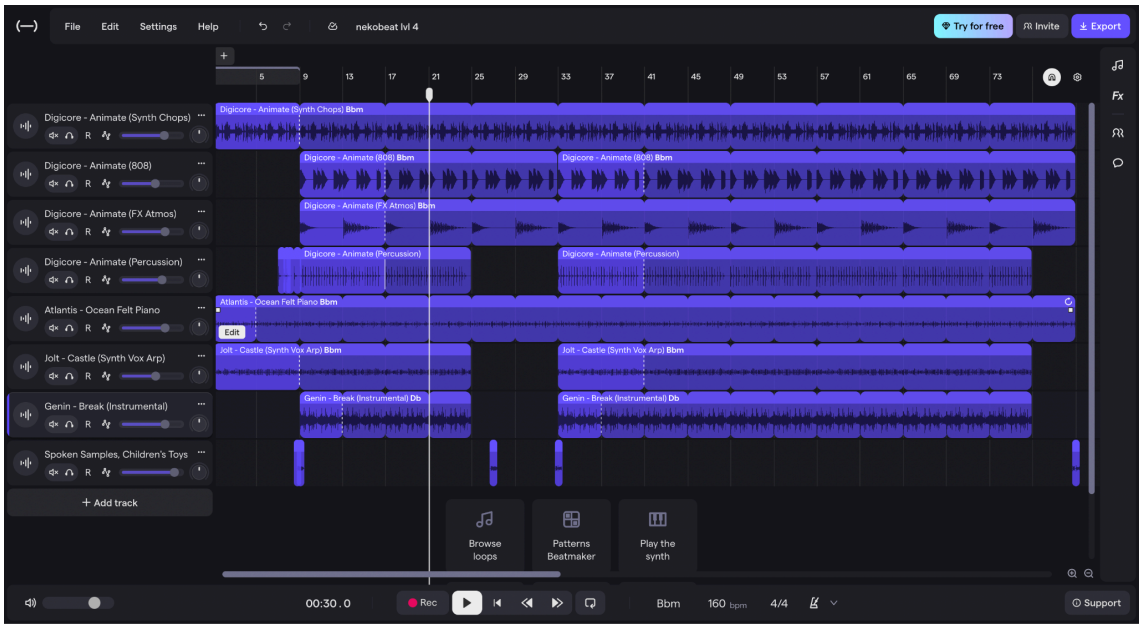
Tools, Resources, or Inputs Used

- Soundtrap (audio production)
- Team discussion and ideation sessions
- Playtesting feedback

GenAI Documentation

No AI was used.





Entry #5: Disability-Framed Iteration + Playtesting

Name: Estelle Kim

Role(s): Lead Experience Designer, Playtesting & Iteration Assistant

Primary responsibility for this work: Ensured the game's three levels effectively communicate Parkinson's disease through gameplay mechanics. Audited mechanics focus on translating research into player experience. Supported playtesting analysis and iteration.

Goal of Work Session

- Refine the player experience across all three levels, ensuring each level builds on the previous one
- Evaluate whether gameplay mechanics (timing, moving bar, screen shake) effectively reflect aspects of Parkinson's
- Support playtesting by observing player interactions and identifying points of confusion, frustration, or engagement
- Help translate playtesting feedback into concrete improvement decisions for iteration

Tools, Resources, or Inputs Used

- ChatGPT 5.2

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: April 2, 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use: To refine how Parkinson's symptoms could be translated into gameplay mechanics and to help interpret playtesting feedback

Summary of Interaction: Asked ChatGPT for ways to represent unpredictability and timing difficulty in a rhythm game and to evaluate whether increasing difficulty meaningfully conveyed the disability experience. Also used it to help organize playtesting observations into clear iteration points

Human Decision Point(s): Only utilized and further built upon ideas that aligned with our teaching outcome and course concepts, rejecting suggestions that relied on increasing difficulty without meaningful connection to the disability. Adjusted ideas to fit our specific game mechanics

Integrity & Verification Note: Cross-referenced AI suggestions with lecture material and prior research on Parkinson's disease. Verified all changes through playtesting and team discussion before implementation

Scope of GenAI Use: Used for brainstorming, critique, and interpreting feedback. All final design and representation decisions were made collaboratively by the team

Limitations or Misfires: Some responses were too general or emphasized solutions based on increasing level difficulty, requiring refinement to better align with our goal

Entry #6: Technical Programmer and Early Implementation

Name: Jasmine Cabrera

Role(s): Lead Programmer (Technical Implementation), Assistant Systems Designer

Playtesting the game extensively, documenting feedback, and supporting the transition from early implementation to a finalized rhythm game concept. I also helped present the game during in-class presentations and contributed to refining the overall gameplay experience

Goal of Work Session

The goal of this phase was to evaluate our early implementation and refine the game based on playtesting results. During development, we initially started implementing a different game concept, but after testing and group discussions, we decided to shift toward a more focused and cohesive rhythm game. Another goal was to improve the player experience by identifying what worked and what did not through repeated playtesting. This included observing how players interacted with the mechanics, pacing, and difficulty of the game. Additionally, we aimed to prepare and present our game to others, which helped us gather further feedback and validate our design direction.

Tools, Resources, or Inputs Used

- Playtesting sessions (individual and group-based)
- In-class presentations and demonstrations
- Peer and instructor feedback
- Team discussions and iteration sessions
- Exist game build and codebase

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

No AI was used during this stage. The work focused on hands-on playtesting, iteration, and presenting the game.

Summary of Process (Human + Tool)

During this stage, our group spent a significant amount of time playtesting the game to better understand how users experienced it. Through this process, we realized that our

initial implementation lacked clarity and consistency in its mechanics. Based on this insight, we made the decision to pivot and fully commit to a rhythm-based game structure.

Decision Points & Trade-offs

- From the start of this project we had to change the core game concept
    - We decided to move away from our initial idea and focus on a rhythm game. While this meant reworking parts of the implementation, it resulted in a more cohesive engaging experience
- Balancing iteration with time constraints
    - We wanted to continue refining the game, but had to balance improvements with project deadlines. This led us to prioritise key gameplay elements rather than overcomplicating features.

Verification & Judgement

- Extensive playtesting
- Group in person/Online discussion
- Peer or instructor feedback
- Comparison with course concepts and assignment criteria

Limitations, Dead Ends, or Open Questions

One limitation during this tate was the need to adjust previously implemented code after changing the game direction. This required additional time and effort to realign the technical structure with the new rhythm-based mechanics.

Appendix

Joanne Lang

ChatGPT

NekoBeat Files Received:
https://chatgpt.com/share/69dba8d4-0dd0-83ea-91cf-8b9f7b6219ff
Miss Feedback Update:
https://chatgpt.com/share/69dba90d-5280-83ea-bdaa-2224cd778553
Rhythm Game Hold Bug Fix:
https://chatgpt.com/share/69dba925-3530-83ea-b35e-e31fa2dcfa0f

Claude AI

NekoBeat rhythm game files:
https://claude.ai/share/ef90342a-6fea-4795-b5bb-226e551a5300
Hold note visual and feedback fixes:
https://claude.ai/share/6828625b-9b9a-45c8-8482-c25690749d68
NekoBeat miss feedback system:
https://claude.ai/share/1c86f3ba-6f38-4494-808e-c1804337f274

Nat Nguyen

```
# Keyboard instruction clarity
_Exported on 4/1/2026 at 17:17:54 EDT from Cursor (2.6.19)_

---

**User**

In Instructions, make it more clear what keys to press, like put a box around the F
and J to illustrate that they're keys actually let's have a diagram or something

---

**Cursor**

Searching the project for where instructions are rendered so we can add clearer key
visuals.



Adding a keyboard-style diagram with boxed F and J keys and matching CSS so it fits
the game's cyberpunk aesthetic.
```

Here’s what changed:

Updates

1. **`index.html`**
  - Replaced the single paragraph with a short block of text plus a **keyboard strip** figure: D-G-H-K as dim “neighbor” keys and **F** and **J** as bright, bordered keys (like physical keycaps).
  - Added a **caption** under the row: “Left hand · **F** · Right hand · **J**” using small inline boxed `<kbd>` elements so F and J read clearly as keys.
  - Wrapped the copy in a `<div class="instructionsText">` with two `<p>` tags so the diagram sits between the intro and the “try to time…” line.
  - Text now says notes match **lanes**, mentions that later levels add keys, and that you **start with F and J** so the diagram matches the beatmaps.

2. **`style.css`**
  - **`.keyDiagram`**: dark panel with a light border so it reads as a small “keyboard” diagram.
  - **`.keyCap` / `.keyCapActive` / `.keyCapDim`**: 3D-style key boxes; F and J use cyan borders and glow (aligned with your lane colors).
  - **`.keyCapInline`**: compact boxes for F and J in the caption.
  - **`.instructionsText`**: `max-width` and paragraph spacing so the layout stays readable.

If you want the diagram to show only **F** and **J** (no D/G/H/K), say so and we can drop the dim keys or replace them with a simpler “two-key” layout.

User

For Level Complete screens, specify which level they completed (1, 2, or 3), and their score with the appropriate score graphic (out of how many stars)

Cursor

Exploring how level completion and scoring work and which score assets exist.

Implementing level-specific titles, star count (matching `getScoreImage`), and a win line for level 3. Checking HTML structure and adding CSS for the new elements.

Adding `getStarCount` to mirror the score-image thresholds, updating the level-complete UI (title, score, stars, win line), and adjusting HTML/CSS.

Here’s what was implemented:

Behavior
- **Title** is always **`LEVEL 1 COMPLETE!`**, **`LEVEL 2 COMPLETE!`**, or **`LEVEL 3 COMPLETE!`** (uses the level you just finished).
- **Extra line** under the title only on level 3: *“You cleared every level — you win!”* (replaces the old generic “YOU WIN” title so the heading stays level-specific).
- **Score graphic** still uses `score0.png` … `score3.png`, driven by a single helper **`getStarCount(score, possible)`** with the same rules as before (≤10% → 0 stars, ≤50% → 1, ≤90% → 2, else 3). **`getScoreImage`** now picks the file from that star count so the image and text never disagree.
- **Copy** under the image:
  - `Score: X / Y` (points vs max for that level)
  - `N out of 3 stars` on the next line.

Files
- **`sketch.js`**: `getStarCount`, `getScoreImage` uses it, `showLevelComplete` sets title, win line, score block (two lines), and image `alt` (`Rating: N out of 3 stars`).
- **`index.html`**: Order is title → win line → image → score block; image uses class `levelCompleteScoreImg` instead of inline max-width.
- **`style.css`**: Styles for the level-complete layout and the two score lines.

If you want the old big **“YOU WIN!”** back as the main headline on level 3, we can swap that in while keeping “Level 3” in the subtitle.

User

Specify which level it is on screen during the gameplay. and Change background images of all the level completed screens and game over screens as well as level 2 and 3 to "bkgdGame2.png"

Cursor

Exploring how levels, backgrounds, and any game-over flow are implemented.

Adding a level badge during gameplay, applying `bkgdGame2.png` to the level-complete screen, game-over screen, and game screen for levels 2–3, and implementing a game-over screen with fail logic. Fixing the retry handler so `calculateLevelEndTime` runs after restoring the level.

Implementing: adding `gameOverScreen` to `showScreen`, `updateGameplayLevelUI`, `showGameOver` (fail when penalties reduce score to 0 after it was positive), and fixing the retry handler.

Here’s what was implemented:

Level during gameplay
- Added a **`gameLevelBadge`** next to **BACK** in the top bar (Jersey 25). It shows **`Level 1`**, **`Level 2`**, or **`Level 3`** and updates whenever the active level changes (`resetGame`, next level, try again, game-over retry, fixed in-game retry).

Background `bkgdGame2.png`
- **CSS** (`.screen.screen--bkgd2`) sets `background-image` to `assets/images/bkgdGame2.png` (same `center/cover fixed` as the base `.screen` rule).
- **Level complete** (`#levelCompleteScreen`): classes `screen screen--bkgd2`.
- **Game over** (`#gameOverScreen`, new): classes `screen screen--bkgd2`.
- **Gameplay** (`#gameScreen`): class `screen--bkgd2` is toggled in **`updateGameplayLevelUI()`** when `level >= 2`; level 1 keeps **`bkgdGame.png`**.

Game over screen
- There was no game-over UI before; there is now a screen with **GAME OVER**, **Level *n***, **Score: *x***, **TRY AGAIN** (same level, full restart), and **HOME**.
- **Failure rule:** if you **earn points** and then **empty-key penalties** bring **score to 0**, the run ends and this screen appears (so you can't fail before you've scored).

Other fix
- **Try Again** on the game screen (hidden retry) now **restores the current level's timing** by calling **`calculateLevelEndTime()`** after putting `level` back (this was wrong before when `resetGame()` forced level 1 during that call).

Add **`assets/images/bkgdGame2.png`** to the repo if it isn't there yet; the paths match your existing `bkgdGame.png` layout.

User

before moving onto level 2 and 3, add instructions like the others saying to press the keys "k" and "d" and space bar for the other keys, as well as for the final screen after finishing level three instead of saying level 3 complete can you do "gameover". and change the backgrounds of all the playing levels, like 2 and 3 to the original bkdgGame.png

---

Cursor

Reviewing the level structure and current UI flow so we can add level intros, adjust the level-3 complete title, and revert gameplay backgrounds.

Implementing the level-intro screen (D/K for level 2, space for level 3), setting the final completion title to "GAME OVER", removing `bkgdGame2` from gameplay, and adding `getHintMessageForLevel` plus `beginLevel`.

Adding the level-intro HTML, CSS for new key styles, fixing the retry handler to update the hint message, and removing any leftover `screen--bkgd2` from the game screen.

Here’s what was implemented and how it behaves.

Level intros (before levels 2 & 3)
- **NEXT LEVEL** no longer jumps straight into play. It opens a **`levelIntroScreen`** with the same kind of layout as the main instructions (title, copy, key diagram, **CONTINUE**).
- **Level 2:** Explains **D** and **K** for the outer (pink) lanes and **F** / **J** for the cyan lanes, with a row **D · F · · · J · K** (dot between F and J for the gap).
- **Level 3:** Explains the **space bar** for the wide yellow center lane and that all five lanes (**D, F, space, J, K**) are used, with a five-key row including a wide **Space** cap.
- **CONTINUE** calls `beginLevel(2)` or `beginLevel(3)` so the correct level starts with reset score and music.

Final win screen
- After **level 3**, the big title is **`GAME OVER`** instead of **`LEVEL 3 COMPLETE!`**.
- The line *"You cleared every level — you win!"* still shows under it so it reads as a win, not a loss.

Backgrounds during play
- **Levels 1–3** all use the normal **`bkgdGame.png`** screen while playing. The code no longer adds `screen--bkgd2` on the game screen, and **`updateGameplayLevelUI()`** always removes `screen--bkgd2`** so an old session can’t leave the wrong background.

Hint line under the top bar
- **`getHintMessageForLevel()`** updates the center hint by level (F/J → D,F,J,K → full five including space). Resets, try again, and retry use the same helper.

Files touched
- `sketch.js` — `beginLevel`, `showLevelIntroScreen`, intro flow on **NEXT LEVEL**, final title logic, gameplay background cleanup, hint messages.
- `index.html` — `#levelIntroScreen`, default message text aligned with level 1.
- `style.css` — styles for outer / space keys, five-key row, level intro content, **CONTINUE** button grouped with other menu buttons.

UI ideas (optional)

1. **Two “GAME OVER” screens** — The win screen and the **lose** screen both say “GAME OVER” in the title area. Consider renaming the **failure** screen to something like **“YOU LOSE”** or **“RUN ENDED”** so the final **GAME OVER** win stays clear.
2. **Score while playing** — A small **current score** (and maybe combo) under the canvas or in the top bar helps players see penalties and progress without waiting for the end screen.
3. **Pause overlay** — A short **“PAUSED”** label on the dimmed canvas so it’s obvious the game is frozen, not broken.
4. **Mute / volume** — A simple **music off** control next to pause is nice for classrooms or public play.
5. **Mobile** — Rhythm games are keyboard-first; a short line on the menu that the game is **keyboard-only** avoids confusion on phones.

If you want, we can rename the **lose** screen title only or tweak the level 3 win line (e.g. *“*Thanks for playing”** under **GAME OVER**).

User

can you implement these 3 ideas: Score while playing — A small current score (and maybe combo) under the canvas or in the top bar helps players see penalties and progress without waiting for the end screen.
Pause overlay — A short “PAUSED” label on the dimmed canvas so it’s obvious the game is frozen, not broken.
Mute / volume — A simple music off control next to pause is nice for classrooms or public play.

Cursor

Reviewing the game structure and input handling to add score, combo, pause overlay, and mute.

Implementing HUD (score + combo), drawing "PAUSED" on the canvas overlay, and a music mute toggle next to pause.

Here’s what was added:

Score + combo (under the canvas)
- **`#playHud`** sits between the canvas and the hit-feedback line.
- **Score** shows `Score X / Y` where **Y** is the running max (same as end-of-level logic, grows as notes spawn).
- **Combo** shows `Combo ×N` after any timed hit (Perfect / Early / Late); it resets to **`Combo —`** on a **missed note**, **empty-key penalty**, or **-100**.

Pause overlay
- On top of the existing dim layer, the canvas now draws **bold “PAUSED”** centered in white so it’s obvious the chart is frozen.

Music mute
- **`MUSIC` / `MUTED`** control is in **`topBarRight`**, to the left of **II PAUSE** (Micro 5, same family as pause).
- Toggling sets **`bgMusic.volume`** to **0** or **0.5**; mute state is kept across levels.
- **`aria-pressed`** and **`aria-label`** are updated for accessibility; **Enter** / **Space** on the control toggles mute.
- **`playBackgroundMusic()`** and the pause handler call **`applyMusicVolume()`** so volume stays correct when resuming.

Files touched
- **`index.html`** — `topBarRight` with `#muteButton`, `#playHud` with `#playScoreDisplay` / `#playComboDisplay`.
- **`style.css`** — `.topBarRight`, `.topBarIconButton`, `.playHud`, combo color.
- **`sketch.js`** — `combo`, `musicMuted`, `updatePlayHud`, `applyMusicVolume`, `updateMuteButtonVisual`, combo rules, pause text, HUD refresh when the game screen is visible.

Note: Only **background music** is muted; **hit beeps** still play so timing feedback stays audible. Say if you want a single control to mute SFX too.

User

Make the background music for the home/instructions/level complete screens chill.mp3

Cursor

Checking how audio and screen navigation are set up.

Adding a second audio element for `chill.mp3` and switching playback in `showScreen`, keeping gameplay on `starstruck.mp3`.

Here’s what changed:

`index.html`
- Added a second `<audio id="menuMusic" src="assets/audio/chill.mp3" preload="auto">` for menu screens.
- **`starstruck.mp3`** stays on **`#bgMusic`** for gameplay only.

`sketch.js`
- **`menuMusic`** reference, **`loop = true`** in `setup()` (same as gameplay track).
- **`applyMusicVolume()`** now sets volume on **both** `bgMusic` and `menuMusic`, so **MUSIC / MUTED** still applies everywhere.
- **`showScreen()`**:
  - **Home, Instructions, Level complete** → pause `bgMusic`, then play **`chill.mp3`** (`menuMusic`).
  - **Game, level intro, game over** → pause **`menuMusic`**; gameplay still uses **`playBackgroundMusic()`** for `starstruck.mp3`.
- **On first load**, if the game screen is hidden (default menu), it tries to start **`menuMusic`** so chill can play on the home screen when the browser allows autoplay.

Add the file **`assets/audio/chill.mp3`** if it isn’t in the repo yet.

User

```
the chill music should only be played during home, instructions, and level complete
screens, not the gameplay screens, right now its playing over the other music during
gameplay, fix this

---

**Cursor**

Tracing audio logic to find why menu music continues during gameplay.



Stopping menu music definitively: pausing may not be enough if `play()` resolves
asynchronously or both tracks were left active. Adding `stopMenuMusic()` (pause +
`currentTime = 0`) and calling it from non-chill `showScreen` paths and at the start
of `playBackgroundMusic()`.



---
```

CHATGPT:
https://chatgpt.com/share/69cd8ddd-f41c-8332-8f18-b33e262f8e52

CHATBOX 1:
https://app.chatboxapp.ai/my/chats/shared/81bb8a5a-2863-43f2-9c40-6076e5a76456

CHATBOX 2:
https://app.chatboxapp.ai/my/chats/shared/4a739446-d2e3-42a1-a5a7-c8f0e8d3be62

CLAUDE AI:
https://claude.ai/share/951f8db1-8238-432d-bbe1-112cc4d3aa3e

Codex
codex://threads/019d4c27-7390-7a81-ba77-ee9600775291

Estelle Kim

Input:

Attached A3 outline, A2 process doc, and A2 group charter

We are a group of five students in GBDA 302: Global Digital Project 2 at the University of Waterloo. Our third assignment (A2) is to build upon our Mid-Term Game, this project requires our group to produce a complete and polished three-level p5.js game that fosters empathy and understanding around a disability-related theme.

We're sharing the A3 assignment outline and our process doc for A2 with you so you understand the context, deadlines and expectations.
Along with that, we are also sharing our group charter from assignment 2.
Using that context, do not create a whole new group charter, but instead update our existing group charter for A2 filling out additional jobs and responsibilities.
Suggest:
• Primary and shadow roles that fit each person's skills

Output (ChatGPT 5.2):

Great—this is a solid charter already. You don't need to reinvent it; you just need to **extend it for A3**, where the focus shifts from "building a prototype" → **polish, 3 levels, cohesion, and playtesting-driven iteration** .

Below is a **clean update to your existing roles** with:

- Added **A3-specific responsibilities**
- Clearly defined **primary vs. shadow expectations**
- Better alignment with what each person already demonstrated in A2 (so it still feels realistic and fair)

🔧 UPDATED ROLES & RESPONSIBILITIES (A3 ADD-ON)

1. Estelle — Lead Experience Designer

Shadow: Playtesting & Iteration Assistant

✅ Primary (UPDATED)

- Refine **core teaching outcome** (what empathy insight players should leave with)
- Ensure **all 3 levels progressively communicate Parkinson's experience**
- Audit mechanics to avoid:

    - oversimplification
    - “difficulty = empathy” trap
- Lead **experience consistency across levels** (not just individual mechanics)
- Collaborate with systems + programming to ensure mechanics reflect:
    - unpredictability
    - timing difficulty
    - motor control challenges

🔁 **Shadow (EXPANDED)**

- Support playtesting analysis by:
    - identifying *why* players feel confused/frustrated
    - linking feedback → empathy goals
- Help write **Iteration Notes (Post-Playtest)** for README

👉 Why this fits: You already led research + framing—now you’re making sure the *final game actually delivers that insight.*

---

2. Ellis — Playtesting & Iteration Lead

Shadow: Assistant Experience Designer

✅ **Primary (UPDATED)**

- Design and run **A3 playtesting session (March 26)**:
    - observation plan (what to watch for)
    - quick note-taking structure
- Identify **3 concrete improvements post-playtest** (required for README)
- Track:
    - player confusion points
    - timing difficulty
    - engagement vs frustration balance
- Coordinate iteration priorities across team

🔁 **Shadow (EXPANDED)**

- Help Estelle evaluate:
    - whether mechanics *feel meaningful*, not just functional
- Ensure feedback connects to **empathy + representation**, not just gameplay

👉 Why this fits: You already handled playtesting + audio—this formalizes you as the **decision bridge after testing**.

---

3. Jasmine — Lead Programmer (Technical Implementation)

Shadow: Assistant Systems Designer

✅ **Primary (UPDATED)**

- Ensure **stable, clean architecture across all 3 levels**
- Handle:
  - bug fixing
  - performance optimization
  - input reliability (CRITICAL for rhythm game)
- Integrate:
  - level transitions
  - shared systems (timing, scoring, feedback)
- Final **GitHub readiness**:
  - file structure (A3 requirements)
  - no breaking errors on load

🔁 **Shadow (EXPANDED)**

- Support Joanne in implementing:
  - new mechanics for Level 2 & 3
- Help translate complex mechanics into **clean, modular code**

👉 Why this fits: You built the foundation—now you ensure the whole game actually *runs smoothly and reliably*.

---

4. Joanne — Lead Systems Designer

Shadow: Assistant Programmer

✅ **Primary (UPDATED)**

- Design and implement:
  - **at least one NEW mechanic** (A3 requirement)
  - difficulty progression across 3 levels

- Balance:
  - hit timing
  - note density
  - mechanic complexity
- Ensure **mechanics evolve meaningfully**, not just “get harder”

Examples of your responsibility:

- Level 1 → basic rhythm + timing
- Level 2 → introduce disruption (e.g., shifting hit zones)
- Level 3 → combine mechanics + unpredictability

🔁 **Shadow (EXPANDED)**

- Support Jasmine with:
  - debugging gameplay logic
  - integrating systems cleanly

👉 Why this fits: You already handled mechanics + iteration—this makes you responsible for **gameplay depth and progression**.

---

5. Nat — Lead Visual & Interaction Designer

Shadow: Assistant Programmer

✅ **Primary (UPDATED)**

- Polish **UI clarity + feedback systems**:
  - hit feedback (visual clarity is HUGE for rhythm games)
  - error states (miss vs success clarity)
- Ensure **accessibility of visuals**:
  - readable contrast
  - clear affordances (what players can/should do)
- Maintain **visual consistency across 3 levels**
- Align visuals with:
  - emotion (stress, pressure, rhythm disruption)
  - cyberpunk aesthetic already established

🔁 **Shadow (EXPANDED)**

- Assist with:

    - UI implementation in code
    - debugging visual alignment issues

👉 Why this fits: You already built the visual identity—now you **turn it into clarity + usability**, which is heavily graded.

🔄 NEW CROSS-ROLE RESPONSIBILITIES (IMPORTANT FOR A3)

These weren't explicit in A2 but are **critical now**:

🧪 Playtesting (ALL MEMBERS)

- Attend + actively participate (mandatory)
- Each member:
    - observe at least 1–2 players
    - note confusion, hesitation, frustration

🧩 Iteration Notes (REQUIRED DELIVERABLE)

- Ellis (lead) + Estelle (support) + Joanne/Jasmine (technical input)
- Must produce:
    - **3 specific changes**
    - tied directly to playtesting

🚀 Final Integration (LAST WEEK PUSH)

- Jasmine + Joanne → functionality
- Nat → UI + feedback polish
- Estelle + Ellis → experience + clarity check

⚖️ Optional (but strong improvement): Micro Role Adjustments

If you want to slightly rebalance workload for A3:

- **Ellis → add "Audio Polish Lead"**

    - refine BGM transitions between levels
    - ensure sound supports gameplay clarity
- **Nat → add "Accessibility Checkpoint Owner"**
    - quick audit: can a new player understand what to do *without explanation*?

🔑 Big Improvement Over Your A2 Charter

What you had before:

- Stage-based roles (good for building)

What you now have:

- **Outcome-based roles (needed for A3 grading)**:
    - empathy ✔
    - polish ✔
    - playability ✔
    - progression ✔

If you want, I can:

- Turn this into a **clean formatted section you can paste directly into your charter**
- Or help you **justify role changes for your Week 8 GenAI reflection (which they're grading you on)**

Great—this is a solid charter already. You don't need to reinvent it; you just need to **extend it for A3**, where the focus shifts from "building a prototype" → **polish, 3 levels, cohesion, and playtesting-driven iteration** .

Below is a **clean update to your existing roles** with:

- Added **A3-specific responsibilities**
- Clearly defined **primary vs. shadow expectations**
- Better alignment with what each person already demonstrated in A2 (so it still feels realistic and fair)

Input:
How could unpredictability or bradykinesia in Parkinson's be represented in a rhythm game mechanic without being frustrating or unfair

Output (ChatGPT 5.2):

This is a really thoughtful design question—and you're right to worry about fairness. If you directly translate symptoms like unpredictability or bradykinesia (slowness of movement) into gameplay penalties, it can quickly feel punishing rather than meaningful.

The key is to **represent the experience without taking away player agency**. Here are some design approaches that tend to work well:

---

1. "Intent vs Execution" Delay (Bradykinesia)

Instead of making inputs fail, introduce a **slight, variable delay between button press and action**.

- Visualize this delay (e.g., a ripple traveling before the note triggers)
- Let players **anticipate and adapt** rather than react instantly
- Keep it within a predictable range so it's learnable

💡 Why it works: The player still succeeds through skill—just a different kind (timing prediction vs reaction speed).

---

2. Adaptive Timing Windows (Fairness Layer)

If you slow the player down, compensate on the system side:

- Dynamically widen the "perfect/good" hit windows when delays occur
- Use "grace zones" where slightly late hits still feel rewarding
- Consider rhythm "magnetism" (notes subtly snap into alignment)

💡 This keeps the game feeling fair even when control feels off.

---

3. Controlled Unpredictability (Not Random Punishment)

Instead of randomness, use **patterned variability**:

- Introduce “off-beat drift” sections where timing subtly shifts
- Use visual/audio cues to foreshadow changes
- Keep unpredictability **telegraphed, not hidden**

💡 Players should feel “this is tricky” rather than “this is broken.”

---

4. Resistance-Based Input (Physical Metaphor)

Simulate hesitation or slowness through mechanics like:

- Holding inputs longer to “push through” a note
- Gradual buildup meters before actions trigger
- Repeated taps to initiate movement

💡 This turns bradykinesia into an **interactive challenge**, not a passive penalty.

---

5. Dual-State Gameplay (On/Off Phases)

Inspired by real Parkinson’s fluctuations:

- Alternate between “fluid” and “restricted” phases
- In restricted phases: slower tempo, heavier controls
- In fluid phases: normal or even boosted responsiveness

💡 This creates contrast and emotional impact without constant frustration.

---

6. Assistive Mechanics as Core Gameplay

Instead of hiding assistance, make it part of the system:

- “Stabilizers” players can activate (e.g., auto-timing, rhythm guides)
- Resource-based aids (use strategically, not permanently)
- Co-op or AI assistance modes

💡 This mirrors real-world coping strategies in a respectful way.

---

7. Audio-Driven Compensation

Since rhythm is auditory:

- Strengthen beat cues when control becomes harder
- Add metronome layers or haptic feedback
- Let sound guide the player when timing feels off

What to Avoid

- Pure RNG input failure (feels unfair instantly)
- Removing control entirely (player disengagement)
- Punishing score systems without compensation
- Long periods of impaired control with no relief

A Good Design Principle

Think of it like this:

> **“Distortion, not denial.”**

You’re bending the player’s interaction with rhythm—not taking it away. The player should still feel *responsible* for success, just operating under different constraints.

If you want, I can help you turn one of these into a concrete mechanic (like a full rhythm game loop or UI concept for your project).

Input:
Does increasing difficulty in a rhythm game effectively communicate Parkinson’s, or does it risk misrepresentation?

Output (ChatGPT 5.2):

Short answer: **it can help, but on its own it's risky and often misleading.** Simply making a rhythm game "harder" usually communicates *challenge*, not the lived experience of Parkinson's.

Why "more difficulty" can misrepresent

Increasing difficulty in a typical rhythm game usually means:

- Faster tempos
- Tighter timing windows
- Denser note patterns

That implies the player needs **better reflexes and precision**. But Parkinson's (especially bradykinesia and motor fluctuations) is less about skill and more about:

- **Delayed or slowed execution**
- **Inconsistency**
- **Mismatch between intention and movement**

So if difficulty = "go faster, be more accurate," it can accidentally suggest:

> "You're failing because you're not skilled enough,"
> instead of
> "Your body isn't responding the way you intend."

That's where misrepresentation happens.

When difficulty *does* work

Difficulty can be meaningful if it's **redefined**, not just increased.

Instead of:

- Harder = faster + stricter

Try:

- Harder = **less reliable control conditions**

Examples:

- Slight input delay variability (you must anticipate, not react)
- Shifting rhythm anchors (beat feels unstable)
- Temporary “resistance” phases where actions take more effort

Now the difficulty reflects **adaptation under constraint**, which is closer to the condition.

The key distinction

There are two very different player experiences:

- **Traditional difficulty:**
  “I missed because I wasn’t fast/accurate enough.”
- **Experiential difficulty (more appropriate here):**
  “I knew what to do, but executing it felt off—and I had to adjust.”

Only the second one meaningfully communicates Parkinson’s.

How to avoid it feeling unfair

If you *do* increase difficulty in this experiential way, you need balancing systems:

- **Compensation mechanics** (wider timing windows, score forgiveness)
- **Clarity** (players understand *why* something felt off)
- **Consistency within variability** (patterns, not randomness)
- **Moments of relief/control** (not constant impairment)

Without these, players will read the game as broken—not intentional.

A more accurate framing

Instead of thinking:

> “How do I make the game harder?”

Think:

> “How do I make control feel *less dependable*, while keeping success achievable?”

That shift is what turns difficulty into representation rather than frustration.

If you want, I can help you map this directly to your GlowTray-style project or prototype a specific level that communicates this clearly.

Input:
During playtesting, players suggested to increase the difficulty of the whole game because it was too easy. So when the player starts getting a streak/combo, add another feature where the screen (gameplay box) starts shaking, making it difficult for the player to focus on catching the notes. Are these changes meaningful/align with our goal of reflecting Parkinson's disease

Output (ChatGPT 5.2):
Does screen shaking on combos align with reflecting Parkinson's disease?

Yes — and meaningfully so. Here's why this works both mechanically and thematically:

✅ Alignment with Parkinson's Disease

Parkinson's is fundamentally about involuntary movement and loss of motor control. The screen shake on combo streaks is a strong metaphor because:

- It simulates tremor escalation — Parkinson's tremors often worsen with sustained effort or stress. A growing combo = sustained effort, so increasing shake mirrors how tremors intensify under exertion.
- It creates perceptual instability — Parkinson's patients experience difficulty with visual tracking and spatial focus due to tremors. Screen shake forces the player to *feel* that instability.

- It punishes success paradoxically — This is emotionally resonant: the better you do, the harder it gets to continue. This reflects the cruel nature of Parkinson's, where *trying harder* doesn't necessarily lead to better outcomes.

✅ Meaningful Difficulty Increase

It's not arbitrary difficulty — it's thematically motivated difficulty, which is the gold standard for serious/empathy games. Compare:

Arbitrary difficulty	Thematically motivated
Faster notes	Screen shake = tremors
More notes	Visual instability = perceptual impairment
Smaller hit windows	Loss of control escalating with effort

⚠️ One Design Consideration

Make sure the shake scales with combo size rather than switching on/off. A gradual escalation (small tremor at 5x combo → intense shake at 20x+) will feel more authentic to how Parkinson's tremors fluctuate in intensity, and give players a moment of "oh no, I can feel it getting worse" — which is powerful for empathy-building.

References

[9] Charity, AbleGamers. “Video Games and Disability Representation.” The AbleGamers Charity, 2023. https://ablegamers.org/video-games-disability-representation/.

[10] Epidemic Sound. n.d. "User Interface, Beep, Error Tone, Soft." Available at: https://www.epidemicsound.com/sound-effects/tracks/17f72d25-b4a6-4d8f-addc-f446ee8b3cb7/.

[6] Fox, Michael J. “The Michael J. Fox Foundation for Parkinson’s Research.” The Michael J. Fox Foundation for Parkinson’s Research | Parkinson’s Disease, 2019. https://www.michaeljfox.org/.

[7] González, Julia, Leya George, Lidia Miteva, and Aneesha Singh. “Developing Empathy towards Experiences of Invisible Disabilities through Games.” In *EmpathiCH ’23: Proceedings of the 2nd Empathy-Centric Design Workshop*, 1–8. University College London, 2023. https://doi.org/10.1145/3588967.3588976.

[8] Karen Anne Cochrane and David Han. 2026. Week 4 - Part 1. GBDA302 Course Slides. University of Waterloo, Stratford, Canada.

[2] Mayo Clinic. “Parkinson’s Disease.” Mayo Clinic, September 27, 2024. https://www.mayoclinic.org/diseases-conditions/parkinsons-disease/symptoms-causes/syc-20376055.

[1] National Institute of Neurological Disorders and Stroke. “Parkinson’s Disease: Challenges, Progress, and Promise.” www.ninds.nih.gov, 2023. https://www.ninds.nih.gov/current-research/focus-disorders/parkinsons-disease-research/parkinsons-disease-challenges-progress-and-promise.

[3] Parkinson's Foundation. “Movement Symptoms.” Parkinson’s Foundation, 2017. https://www.parkinson.org/Understanding-Parkinsons/Movement-Symptoms.

[4] Parkinson's Foundation. “Non-Movement Symptoms.” Parkinson’s Foundation, 2023. https://www.parkinson.org/Understanding-Parkinsons/Non-Movement-Symptoms.

[5] TEDx Talks. “My Parkinson’s Story: Lessons on Optimism and Opportunity | Mark Colo.” *Youtube*, 2023. https://youtu.be/asf-YXe7hXk?si=2Vrr3nJTfbasGO85.